

Git 版本管理

做出作品後，下一個能力是讓修改過程可追蹤、可合作、可回復；這正是 Git 與 GitHub 要解決的問題。

版本管理：每一次修改都要能回頭看

Git 負責管理本機資料夾的版本；GitHub 是把專案放上網路、分享與協作的平台。

本機工作資料夾 → Git commit → GitHub 遠端 repository → 團隊 clone/pull/push。

- 一次 commit 只完成一個小目的，例如「完成輸入輸出練習」。
- 專案要有 `README`：目的、如何執行、目前完成什麼。

Windows 起點：在同一個專案資料夾操作

假設電腦已安裝 Git，並建立好空資料夾 `my-first-program`。

```
C:\Users\你的帳號\Documents\my-first-program\
```

在 VS Code 開啟這個資料夾，再開啟整合終端機。PowerShell、CMD 或 Git Bash 擇一即可；以下 `git ...` 指令相同。

```
git --version  
git status
```

第一次執行 `git status` 顯示「不是 Git repository」是正常的：**下一步才要初始化。**

第一次使用 Git：先設定作者識別資料

```
git config --global user.name "Your Name"
git config --global user.email "student@example.com"
git config --global init.defaultBranch main
git config --global --list
```

設定	用途
user.name	寫進每一筆 commit 的作者名稱
user.email	寫進 commit 的作者信箱；可使用 GitHub 已驗證信箱或隱私信箱
init.defaultBranch main	之後新專案預設使用 main 作為主幹分支

這些是**版本作者資料**，不是 GitHub 登入密碼；請把範例文字換成自己的資料。

git init：讓空資料夾開始記錄版本

確認終端機目前位於 `my-first-program`，再執行：

```
git init -b main
git status
git branch --show-current
```

- `git init -b main`：建立隱藏的 `.git` 版本資料，並指定主幹為 `main`。
- `git status`：確認目前分支，以及哪些檔案尚未追蹤或已修改。
- `.git` 是版本歷史的核心；**不要把它當成一般資料夾刪除或搬動。**

`git init` 不會把檔案上傳，也不會自動建立 commit；它只讓目前資料夾成為本機 repository。

選讀：若既有 repository 的主幹仍叫 `master`，可用 `git branch -M main` 改名；本頁新建立的專案不需要執行。

建立檔案：先準備專案說明與忽略規則

先在 VS Code 建立三個檔案：

my-first-program/	
—— README.md	← 作品名稱、用途與執行方式
—— .gitignore	← 排除環境、建置與暫存檔
—— hello.py	← 或依課程建立 hello.java

`README.md` 告訴別人「這個專案怎麼使用」；`.gitignore` 告訴 Git「哪些未追蹤檔案不要加入版本」。

在 VS Code 新增檔案時，名稱要完整輸入 `.gitignore`；不要存成 `.gitignore.txt`。

`.gitignore` 也應該 commit，讓 clone 專案的同學共用相同規則。

`.gitignore`：排除不適合進入版本的檔案

在專案根目錄的 `.gitignore` 加入：

```
# Windows 與編輯器
Thumbs.db
.vscode/

# Python 環境與暫存
__pycache__/*
*.pyc
.venv/

# Java 編譯與建置產物
*.class
out/
target/

# 本機環境變數
.env
```

`.vscode/` 有時包含團隊要共用的設定；若老師要求統一格式或除錯設定，就不要整個忽略。

`.gitignore` 不是保密工具。密碼、API key 與個資一開始就不應放進專案或 commit。

先讀懂 `.gitignore` 的四種基本寫法

寫法	意義	範例
#	註解，不是忽略規則	# Python cache
資料夾/	忽略該資料夾下的內容	.venv/
*	比對任意字串	*.class
!	將原本忽略的項目排除在規則外	!keep.txt

規則由上往下套用；可以用 `!` 讓特定檔案重新被納入追蹤範圍。

確認 `.gitignore` 生效，再加入必要檔案

```
git status
git status --ignored
git check-ignore -v .venv
git add .gitignore README.md hello.py
git status
```

`git add` 是把指定變更放進 **staging area (暫存區)**，決定下一筆 commit 要包含什麼；它還不是 commit，也沒有上傳。

`git add .` 會加入目前資料夾下所有變更；新手應先看 `git status`，確認沒有密碼、金鑰或不該公開的檔案。

已經被 Git 追蹤，新增 `.gitignore` 也不會消失

`.gitignore` 只影響尚未追蹤的檔案。若 `out/app.class` 已經 commit，先讓 Git 停止追蹤，再保留本機檔案：

```
git rm --cached out/app.class
git status
git commit -m "chore: stop tracking generated file"
```

接著確認 `.gitignore` 已包含適用規則，例如：

```
*.class
out/
```

若密碼或 API key 已經 commit 或 push，刪除檔案或加入 `.gitignore` 都無法清除歷史。請立刻停用並更換該密鑰，再請老師或維護者協助處理歷史紀錄。

commit：替一個完成的小目的留下版本

```
git commit -m "docs: add project introduction"
git status
```

好的 commit message 要讓人不打開檔案，也能知道這次完成什麼：

類型	範例	代表的目的
功能	feat: add score calculator	加入一項可使用的功能
修正	fix: handle empty input	修正一個明確問題
文件	docs: explain how to run	補充 README 或使用說明

一次 commit 處理一個小目的。避免 update、final、修改一下 等看不出內容的訊息。

log：讀懂自己走過的版本歷史

```
git log --oneline --graph --decorate --all
git show HEAD
git status
```

指令	你要觀察什麼
git log --oneline	每筆 commit 的短編號與訊息
--graph --decorate --all	分支如何分開、合併，以及目前指向哪裡
git show HEAD	最新 commit 實際改了哪些內容
git status	工作區是否還有未提交的變更

commit 是本機版本紀錄；只有 push 之後，GitHub 才會收到這些 commits。

第一次 `push`：把本機 repository 連到 GitHub

先在 GitHub 建立空 repository；已有本機 commit 時，不要勾選建立 README、License 或 `.gitignore`。

```
git remote add origin https://github.com/YOUR-NAME/my-first-program.git
git remote -v
git push -u origin main
```

- `origin`：遠端 repository 的常用名稱。
- `-u origin main`：推送 `main`，並記住本機與遠端分支的對應；之後可直接 `git push`。
- HTTPS 驗證不要輸入 GitHub 帳號密碼；Windows 常由瀏覽器或 Git Credential Manager 完成，也可依課程設定使用 token／SSH。

GitHub 是分享與協作平台；Git 的 commit 歷史仍先存在你的本機 repository。

主幹與開發分支：先分開工作，再決定何時整合

分支	用途	新生先記住
main	可執行、已確認的主幹版本	不要把未完成實驗直接堆進主幹
feature/readme	一項功能或文件工作的短期分支	完成、測試、合併後即可刪除
fix/input-error	修正一個明確問題	名稱要看得出目的
develop	部分團隊使用的整合分支	只有團隊流程要求時才建立

```
git switch -c feature/readme
git branch
git switch main
```

分支是指向 commit 的輕量指標；切換分支會讓工作資料夾呈現該分支的版本。

在功能分支完成一個小修改

```
git switch -c feature/readme
# 在 VS Code 修改 README.md，儲存後繼續
git status
git diff
git add README.md
git commit -m "docs: add learning goals"
git push -u origin feature/readme
```

1. `status`：知道哪些檔案改過。
2. `diff`：提交前逐行確認變更。
3. `add` 與 `commit`：留下可回頭看的本機版本。
4. `push`：讓 GitHub 收到功能分支，可建立 Pull Request 供同學檢查。

分支不是另一份手動複製的資料夾；不要建立 `project-final`、`project-final2` 來代替版本管理。

遠端有更新：先看清楚，再決定如何整合

常見情境：同學已推送新 commit、你在另一台電腦更新過，或 GitHub 上的 Pull Request 已合併。

```
git status
git fetch origin
git log --online --graph --decorate --all
git pull --ff-only
```

指令	作用
fetch	下載遠端的新 commits，但暫時不改目前工作檔案
pull	先 fetch，再把遠端更新整合進目前分支
--ff-only	只接受可直接前進的情況；歷史分岔時先停下來檢查

若 `git status` 顯示尚未提交的修改，先確認、commit 或依教師指示暫存；不要直接反覆 pull 碰運氣。

merge：把完成的分支整合回主幹

確認功能分支已 commit 且測試通過，再回到主幹：

```
git switch main
git pull --ff-only
git merge feature/readme
git push origin main
git branch -d feature/readme
```

- `git merge feature/readme` 的意思是：把該分支可到達的新 commits 整合進**目前所在的 main**。
- 團隊協作時，通常先在 GitHub 建立 Pull Request，經檢查後合併；本機再切回 `main` 並 pull。
- 刪除已合併的短期分支，可以讓分支清單保持清楚；刪除前先確認成果已整合。

Merge conflict：Git 需要你判斷哪個內容才正確

當兩個分支修改同一段內容，Git 可能無法自動合併：

```
git status
# 在 VS Code 開啟衝突檔案，選擇或重新編寫正確結果
git add README.md
git commit
```

處理原則：

1. 先讀懂 current 與 incoming changes，不要盲目選「全部接受」。
2. 刪除衝突標記，執行程式或測試，確認合併後結果。
3. `git add` 標記已解決，再用 commit 完成 merge。
4. 尚未準備好時可用 `git merge --abort` 回到合併前。

clone：從 GitHub 取得一個已存在的專案

在 GitHub 專案頁按 **Code**，複製 HTTPS URL；終端機先移到「準備放專案的上一層資料夾」。

```
cd $HOME\Documents
git clone https://github.com/OWNER/REPOSITORY.git
cd REPOSITORY
git remote -v
git log --oneline
git status
```

git init	git clone
把目前資料夾變成本機 repository	建立新資料夾，下載完整 repository 與版本歷史
適合從自己的空資料夾開始	適合取得 GitHub 上已存在的專案

不要先建立同名資料夾再把檔案逐一下載；clone 才能保留分支、commit 與遠端設定。

版本管理實戰：兩人在 `main` 上協同作業

用三個情境，練習「兩人推同一個 repository」時的實際指令。

前提設定：

- 同學 A、同學 B 都已 `git clone` 同一個 GitHub repository。
- 這一單元**先不使用 branch／fork／Pull Request**（下一單元再學）。
- `origin` 是 GitHub 上的共用 repository，`origin/main` 是它的主幹。

協作黃金守則：① 開工前先 `git pull` ② `git push` 前再 `git pull` 一次 ③ `commit` 小而集中 ④ 要大改某檔案前先在群組講一聲。

情境 1 (A)：多次修改多個檔案，再一次 push

```
# —— 第 1 次修改：改 hello.py 與 README.md ——  
git status  
git add hello.py README.md  
git commit -m "feat: validate user input"  
  
# —— 第 2 次修改：再改 hello.py，並新增 notes.md ——  
git add .  
git commit -m "docs: add week-2 notes"  
  
# —— 送到 GitHub：push 前先確認沒有落後 ——  
git pull --ff-only  
git push
```

- 每次 `commit` 只存在**本機**；GitHub 這時還看不到。
- `git push` 會一次送出「本機有、遠端沒有」的**所有 commit**（此例為 2 筆），不是只送最後一筆。
- `git log --oneline origin/main..HEAD` 可先看「這次會 push 出去哪些 commit」。

情境 1（B）：更新到最新版本，並先比較差異

```
# 只下載遠端更新，先不動我的工作檔案
git fetch origin

# 比較「我的 main」與「遠端 main」
git log --oneline main..origin/main # 遠端比我多了哪些 commit
git diff main origin/main          # 這些 commit 改了哪些內容
git log --graph --oneline --all     # 用圖看歷史怎麼接上

# 確認沒問題，才真的更新工作區
git pull --ff-only
```

指令	作用
fetch	更新 <code>origin/main</code> ，工作區檔案不變
A..B	「B 有、A 沒有」的 commit 範圍
pull --ff-only	只在能直接前進時更新；分岔就停下

情境 2（起）：兩人同時改同一個檔案

```
# 同學 B 先完成並成功 push
git add README.md
git commit -m "docs: rewrite intro section"
git push                                #  成功
```

```
# 同學 A 也改了 README.md，接著想 push
git add README.md
git commit -m "docs: expand intro section"
git push
# ! [rejected]          main -> main (non-fast-forward)
# error: failed to push some refs to '...github.com/...'
# hint: Updates were rejected because the remote contains work
# hint: that you do not have locally. ... integrate the remote changes
```

- 被拒原因：遠端已有一筆 A 本機沒有的 commit，Git 不會讓你直接蓋過去。
- 正確做法：先 `git pull` 把兩邊整合起來，再 `git push`。不要用 `--force`。

情境 2（合 1）：pull 之後 Git 會怎麼做

```
git pull --no-rebase    # 抓遠端更新並嘗試合併
```

情況一：兩人改的是不同段落

→ Git **自動合併**，跳出編輯器要你寫 merge 訊息，直接存檔關閉即可，不需人工判斷。

情況二：兩人改到同一段落

→ 出現 **CONFLICT**，Git 停下來等你決定：

```
git status              # README.md: both modified
```

衝突檔案中會插入標記：

```
<<<<<<< HEAD
(你的版本)
=====
(同學 B 的版本)
>>>>>>> 3f5a1c2...
```

情境 2（合 2）：人工解決衝突並完成合併

```
# 1. 在 VS Code 開衝突檔，刪掉 <<<<<<< 、===== 、>>>>>>> 三行，  
# 留下正確內容（可融合兩人的寫法）  
# 2. 執行程式或測試，確認合併後正確  
git add README.md          # 標記「這個檔案我解決好了」  
git status  
git commit                  # 完成這次 merge（訊息用預設即可）  
git push
```

- 是否需要人工合併？**改到同一段落就需要**；Git 不會猜哪一版才對。
- `git diff` 在解衝突時可看目前狀態。
- 還沒把握、想重來：`git merge --abort` 回到 pull 前。

情境 3 (A)：回復到前面某一個版本

```
# 1. 找出「當時比較好」的那一版
git log --oneline
# a1b2c3d (HEAD -> main) style: 改壞了版面
# 9f8e7d6 refactor: 調整結構
# 3c4d5e6 docs: 這一版最好      ← 想回到這個內容

# 2. 先看清楚差在哪 (不改動任何東西)
git show 3c4d5e6
git diff 3c4d5e6 HEAD
```

推薦做法：用一筆新的 commit 回到舊內容（安全、歷史保留）

```
git restore --source 3c4d5e6 .    # 把工作區內容換成該版本
git status
git add .
git commit -m "revert: 回到 3c4d5e6 的版面設計"
git push
```

課堂**禁止** `git reset --hard` 後 `git push --force`：那會改寫共用歷史，覆蓋同學的紀錄。

情境 3 (B)：另一位如何拿到「回復」這件事

```
git fetch origin
git log --online main..origin/main
# 7a8b9c0 revert: 回到 3c4d5e6 的版面設計 ← A 新增的那筆

git pull --ff-only      # 可直接前進，沒有衝突
```

- 因為 A 是用「**新 commit**」回復內容，對 B 來說就只是「多一筆更新」，正常 `pull` 就拿到了，不需要任何特殊指令。
- 若 A 當初用 `reset --hard` + `push --force`，B 這裡就會看到歷史分岔、`pull` 失敗，得手動救回——這正是課堂堅持用 `restore` / `revert` 的原因。

兩人協作：常見狀況對照表

狀況	主要指令	需要人工判斷？
對方推了新 commit，我要跟上	<code>git fetch</code> → <code>git pull --ff-only</code>	否
比較我和遠端差在哪	<code>git log main..origin/main</code> 、 <code>git diff main origin/main</code>	—
<code>push</code> 被拒 (non-fast-forward)	<code>git pull --no-rebase</code> → 解決後 <code>git push</code>	視情況
兩人改到不同段落	<code>pull</code> 後自動合併，存檔即可	否
兩人改到同一段落	編輯衝突檔 → <code>git add</code> → <code>git commit</code> → <code>git push</code>	是
回到舊版本內容	<code>git restore --source <id> .</code> → <code>commit</code> → <code>push</code>	是 (選保留哪一版)

只要「pull 在先、commit 集中、改大檔前先講」，多數衝突都能避免。

課堂任務：把專案資料夾簽入 GitHub

以 `my-first-program` 完成並繳交：

必做項目	驗收證據
設定作者資料，以 <code>main</code> 初始化 repository	<code>git config --global --list</code> 、 <code>git branch --show-current</code>
建立 README、 <code>.gitignore</code> 與一個可執行程式	GitHub 看得到必要檔案，但看不到環境、暫存或編譯產物
至少完成兩筆目的清楚的 commits	<code>git log --oneline</code> 顯示兩筆不同訊息
建立一個 <code>feature/...</code> 分支並修改內容	分支 commit 與合併紀錄可被說明
將主幹推送到自己的 GitHub repository	繳交 repository 連結

禁止提交：密碼、API key、個資、正式資料或不確定授權的檔案。

完成後請口頭說明：哪一筆 commit 完成什麼？分支如何回到 main？

推薦的 Git 工具：先懂流程，再選介面

工具	適合怎麼用	新生要注意
Git CLI／Git Bash	學會真正的 <code>status</code> 、 <code>add</code> 、 <code>commit</code> 、 <code>pull</code> 、 <code>push</code>	指令最通用；先確認目前資料夾與分支
VS Code Source Control	看 diff、暫存、commit、分支圖與解衝突	使用的是電腦已安裝的 Git；圖形介面不取代理解流程
GitHub Desktop	用圖形介面 clone、commit、push、建立分支與 Pull Request	適合 GitHub 入門；仍要會讀 commit 與 diff
Fork	以視覺方式查看歷史、分支、diff、merge conflict、rebase	Windows／macOS 可用；進階功能等基礎穩定後再學
Git Credential Manager	在 Windows 安全完成 GitHub HTTPS 驗證	不要把 token 或密碼寫進程式、README 或 remote URL

課堂建議：終端機與 VS Code 為主；需要更清楚的歷史圖或衝突介面時，再選 GitHub Desktop 或 Fork。